

Efficient Fixed/Floating-Point Merged Mixed-Precision Multiply-Accumulate Unit for Deep Learning Processors

Hao Zhang ^{*}, Hyuk Jae Lee [†] and Seok-Bum Ko ^{*}

^{*}Department of Electrical and Computer Engineering, University of Saskatchewan, Canada

[†]Department of Electrical and Computer Engineering, Seoul National University, Korea

Email: {hao.zhang, seokbum.ko}@usask.ca, hjlee@capp.snu.ac.kr

Abstract—Deep learning is getting more and more attentions in recent years. Many hardware architectures have been proposed for efficient implementation of deep neural network. The arithmetic unit, as a core processing part of the hardware architecture, can determine the functionality of the whole architecture. In this paper, an efficient fixed/floating-point merged multiply-accumulate unit for deep learning processor is proposed. The proposed architecture supports 16-bit half-precision floating-point multiplication with 32-bit single-precision accumulation for training operations of deep learning algorithm. In addition, within the same hardware, the proposed architecture also supports two parallel 8-bit fixed-point multiplications and accumulating the products to 32-bit fixed-point number. This will enable higher throughput for inference operations of deep learning algorithms. Compared to a half-precision multiply-accumulate unit (accumulating to single-precision), the proposed architecture has only 4.6% area overhead. With the proposed multiply-accumulate unit, the deep learning processor can support both training and high-throughput inference.

I. INTRODUCTION

Deep learning [1] has achieved great success in recent years. However, the hardware cost to implement a deep learning model is high since it contains large amount of parameters and is computationally intensive. Many research works [2] have been done recently in order to reduce the hardware cost. As the core of computation, the functionality of arithmetic unit can determine the functionality of the whole hardware processor. In this paper, we will focus on novel arithmetic unit design to enrich the functionality of deep learning processor.

For the inference operations of deep learning applications, 8-bit fixed-point or even lower precisions have been proved to be sufficient to maintain high accuracy [3] [4]. However, for training operations, larger data range is still required to handle the computation of gradients and back-propagation. Recent research work [5] has reported that 16-bit half-precision operations can successfully train the deep learning models. Therefore, in order to efficiently support both training and inference, a fixed/floating-point merged arithmetic unit is required.

In deep learning applications, a large amount of multiplication results need to be accumulated to obtain a single output. Therefore, a number format with larger range and higher precision is required to maintain the accuracy. This is suggested in [6] and [7]. The results of half-precision

multiplication can be accumulated to single-precision format. Similarly, the results of 8-bit fixed-point multiplication can be accumulated to 32-bit fixed-point format. The accumulation to higher precision can avoid data loss due to rounding. A mixed single/double-precision fused multiply-add unit has been proposed in [8]. However, for deep learning applications, lower precision operations are preferred.

In this paper, a novel fixed/floating-point merged mixed-precision multiply-accumulate unit is proposed to support the above mentioned deep learning training and inference operations. For deep learning training, the proposed architecture supports 16-bit half-precision multiplications and the product will be accumulated to a 32-bit single-precision accumulator. During inference operations, the two 16-bit inputs are filled with two sets of 8-bit fixed-point operands. The proposed architecture will perform two parallel 8-bit multiplications. These two products can be added and then accumulated to 32-bit fixed-point accumulator. The fixed/floating-point merged multiplication is realized by the Karatsuba algorithm [9]. With the proposed arithmetic unit, the deep learning processor can support training operations as well as high-throughput inference operations.

The rest of the paper is organized as follows: Section II presents the IEEE floating-point format and Karatsuba algorithm. The proposed mixed-precision multiply-accumulate unit is presented in Section III. The synthesis results of the proposed architecture and the results analysis are presented in Section IV. Finally, Section V concludes the whole work.

II. BACKGROUND

A. IEEE Floating-Point Format

The floating-point format presented in IEEE 754 standard [10] has three components: sign (s), biased exponent (E), and mantissa (m). The actual exponent value $e = E - bias$. For half-precision numbers, they have 1-bit s , 5-bit E , and 10-bit m . The exponent bias is 15. For single-precision numbers, they have 1-bit s , 8-bit E , and 23-bit m . The exponent bias is 127.

B. Karatsuba Algorithm

The Karatsuba algorithm is proposed in [9]. It is used to divide a large multiplication into several smaller ones. Suppose

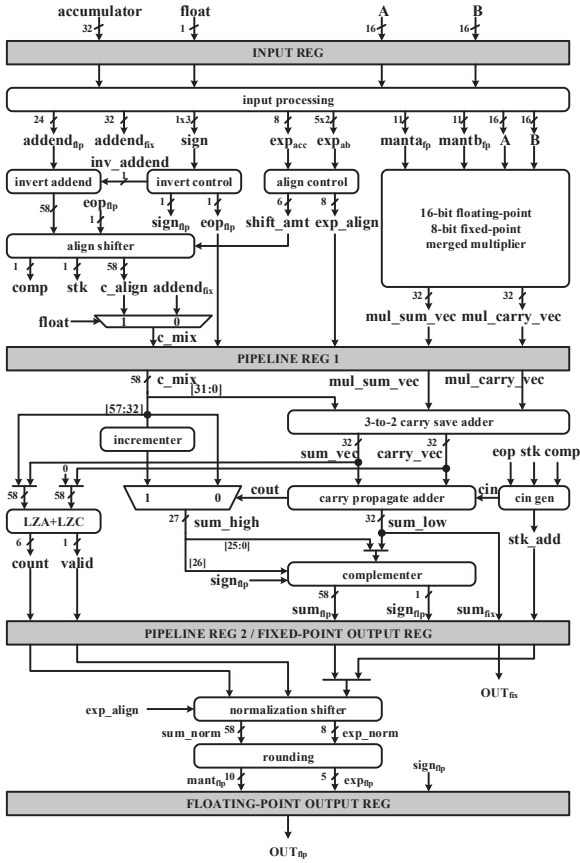


Fig. 1. Data-path of the proposed fixed/floating-point merged mixed-precision multiply-accumulate unit

A and B are two multiplication operands and represented as $A = A_1 2^n + A_2$ and $B = B_1 2^n + B_2$, where $A_2(B_2)$ is the less significant n -bit of $A(B)$ and $A_1(B_1)$ is remaining bits of $A(B)$. In general, the multiplication of A and B can be calculated as follows:

$$\begin{aligned} A \times B &= (A_1 \times 2^n + A_2)(B_1 \times 2^n + B_2) \\ &= A_1 B_1 \cdot 2^{2n} + (A_1 B_2 + A_2 B_1) \cdot 2^n + A_2 B_2 \end{aligned} \quad (1)$$

In Karatsuba algorithm, the term $A_1 B_2 + A_2 B_1$ is rewritten as:

$$A_1 B_2 + A_2 B_1 = A_1 B_1 + A_2 B_2 - (A_1 - A_2)(B_1 - B_2) \quad (2)$$

Therefore, only three multiplications instead of four ($A_1 \times B_1$, $A_1 \times B_2$, $A_2 \times B_1$, and $A_2 \times B_2$) are enough to calculate $A \times B$. However, as shown in equation (2), the algorithm requires additional adders.

III. THE PROPOSED ARCHITECTURE

The data-path of the proposed fixed/floating-point merged multiply-accumulate unit is shown in Fig. 1. It is similar to a standard fused multiply-add (FMA) architecture [11], except the third operand (*accumulator*) is in higher precision. The

whole design contains three pipeline stages. The first pipeline stage is for low-precision multiplication and alignment of the high-precision accumulator. The second pipeline stage is for the accumulation operation and leading zero detection for floating-point operations. The last pipeline stage is for floating-point normalization and rounding operation. The floating-point operation requires the whole three pipeline stages while the fixed-point operation requires the first two pipeline stages.

A. Floating-Point Mode

The proposed design accepts a control signal *float* which is to control whether floating-point operation or fixed-point operation is performed. In floating-point (FLP) mode, A and B are two half-precision numbers and *accumulator* is a single-precision number. The proposed architecture can accomplish $A \times B + \text{accumulator}$ operation.

The mantissa of A and B , manta_{fp} and mantb_{fp} , are extracted and sent to the merged multiplier to generate the carry-save format result. In half-precision multiplication, 22-bit is enough for the product. However, in order to support 32-bit accumulation in fixed-point mode, the products are extended to 32-bit by padding zeros to the right of the actual products. At the same time, the accumulator is inverted in case of subtraction ($\text{eop}_{fp} = 1$) and then aligned to be ready for accumulation. Similar to the FMA design in [12], the *accumulator* is placed two bits to the left of the carry-save format products. This is to ensure there will be no overflow when accumulating and the *accumulator* only needs to be shifted to the right. In the proposed design, a 58-bit shifter is required. When generating the shifting amount, the exponents of A and B are first converted to single-precision exponent by adding the bias difference of half-precision and single-precision.

In the next stage, the lower 32-bit of the aligned *accumulator* is combined with carry-save format products and then go through the carry-propagate adder to generate the lower part accumulation result. The higher part of the aligned *accumulator* is incremented and the output carry of the lower carry-propagate adder determines whether incremented *accumulator* is chosen. In parallel, an leading-zero anticipator and counter [13] is used to generate the shifting amount for normalization shifter.

In the last stage, the accumulation result is normalized and rounded to single-precision mantissa bit-length. The sign, exponent, and mantissa are combined to form the IEEE 754 single-precision format.

B. Fixed-Point Mode

In fixed-point (FIX) mode, A (B) contains two sets of 8-bit fixed-point numbers A_h, A_l (B_h, B_l). The *accumulator* contains one 32-bit fixed-point number. The proposed design can perform $A_h \times B_h + A_l \times B_l + \text{accumulator}$ operation.

Fixed-point numbers are in two's complement format. Therefore, inversion is not required for *accumulator*. In addition, the alignment is also not required and the 32-bit *accumulator* is directly put to the lower 32-bit position.

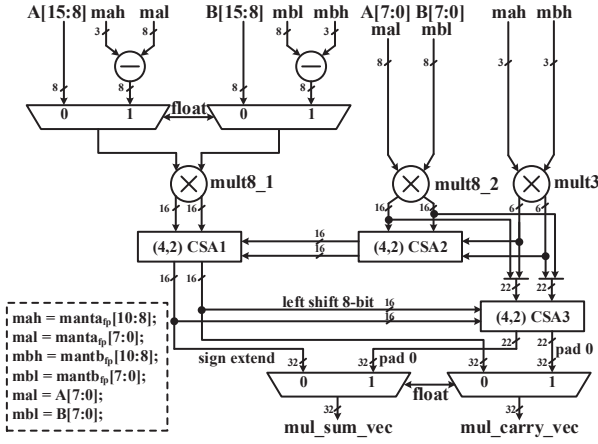


Fig. 2. Architecture of the proposed fixed/floating-point merged multiplier

Moreover, complementer in the second stage is not required since the fixed-point is in two's complement format.

C. Merged Multiplier Design

The architecture of the merged FLP and FIX multiplier is shown in Fig. 2. It contains two 8-bit multipliers, *mult8_1* and *mult8_2*, and one 3-bit multiplier *mult3*. Two 8-bit multipliers, *mult8_1* and *mult8_2*, are used for two parallel multiplications in FIX mode. These two products are accumulated by (4,2)CSA1 and then the results are sign extended to 32-bit to form the final multiplier output.

In FLP mode, the $manta_{fp} \times mantb_{fp}$ is calculated with the Karatsuba algorithm. Only one more 3×3 multiplier, in addition to these two 8-bit multipliers, is enough to generate half-precision product. $manta_{fp}$ and $mantb_{fp}$ are rewritten as:

$$\begin{aligned} manta_{fp} &= mah \cdot 2^8 + mal \\ mantb_{fp} &= mbh \cdot 2^8 + mbl \end{aligned} \quad (3)$$

Then according to the Karatsuba algorithm, $mah \times mbh$ is calculated by *mult3*, $mal \times mbl$ is calculated by *mult8_2*, and $(mah - mal) \times (mbh - mbl)$ is calculated by *mult8_1*. Here, an addition in (4,2)CSA1 is required to accumulate two products in FIX mode. Therefore, we modify the last term in equation (2) to $(A_1 - A_2) \times (B_2 - B_1)$, so that the subtraction of this term will become addition. Therefore, *mult3* here calculates $(mah - mal) \times (mbh - mbl)$. (4,2)CSA2 will accumulate $mah \times mbh$ and $mal \times mbl$. The accumulated result will be combined with $(mah - mal) \times (mbh - mbl)$ by (4,2)CSA1 to generate $mah \times mbl + mal \times mbh$. Finally, $mah \times mbl + mal \times mbh$ is shifted by 8-bit and $mah \times mbh$ is shifted by 16-bit. These two together with $mal \times mbl$ are combined by (4,2)CSA3 to generate the final carry-save format product.

Since *mult3* is used only for FLP mode, it is an unsigned multiplier. *mult8_1* and *mult8_2* are used in both FLP and FIX mode. In FLP mode, an unsigned multiplier is required while in FIX mode, a signed multiplier is required. The

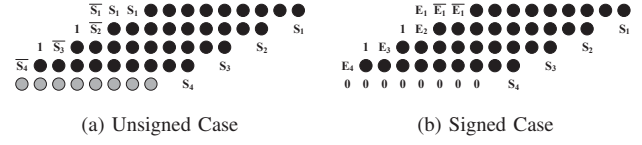


Fig. 3. Partial products arrangement of modified booth multiplier

TABLE I
SYNTHESIS RESULTS OF THE COMBINATIONAL LOGIC OF EACH PIPELINE STAGE

Pipeline	Delay (ns)	Area (μm^2)	Power (mW)
Stage 1	0.74	20003.75	7.5
Stage 2	0.70	7958.69	1.87
Stage 3	0.68	4227.95	0.63

modified booth multiplier [14] is used for two 8-bit multiplier. The algorithm is modified to support both signed and unsigned multiplication.

The partial product arrangement of signed and unsigned booth multiplier is shown in Fig. 3. The dots are either $\pm$ multiplier or $\pm 2 \times$ multiplier. For unsigned case, the multiplier is always extended with zero and in signed cases, the multiplier is sign extended. This can be handled by the control signal *float*. The partial product generate logic is same for signed and unsigned cases. So the *S* bit which is the complement bit in the case of negative partial product is used in common. In order to support signed cases, the *E* bit, which is used to determine whether the partial product and multiplier have the same sign, is also generated. When extending $\pm 1 \times$ or $\pm 2 \times$ multiplier, as shown in Fig. 3, whether to use *S* bit or *E* bit can be handled by *float*.

The unsigned case has one more partial product than the signed case. However, in 8-bit case, the most significant two bits of the last group of multiplier bits to generate partial product is always "00". Therefore, the last partial product is either zero or $1 \times$ multiplier. We do not need partial product selector to generate the last partial product. In addition, in signed case, the *float* control can be used to set the last partial product all zeros.

IV. RESULTS AND ANALYSIS

The proposed design is implemented with VHDL. Simulations with extensive testing vectors are performed to verify the functionality of the proposed design. The proposed design is then synthesized with STM-90nm technology with normal case parameters using Synopsys Design Compiler.

The combinational logic of each pipeline stage is first synthesized. The synthesis result of each pipeline stage is shown in Table I. The critical path is in the first pipeline stage where the time consuming multiplier is implemented. The first stage also has the largest area. The multiplier is area consuming and alignment shifter also occupies large area.

TABLE II
COMPARISON OF THE PROPOSED MULTIPLY-ACCUMULATE UNIT WITH SINGLE-MODE MULTIPLY ACCUMULATE UNIT

	Latency	Throughput (GOPS)	Worst Delay		Area		Power (mW)
			ns	FO4*	μm^2	NAND2*	
Fixed-Point MAC [†]	2	1.43	0.7	15.56	13257.91	3013	6.65
Floating-Point MAC [‡]	3	1.25	0.8	17.78	40817.54	9276	13.59
Proposed MAC	2 (Fixed-Point) 3 (Floating-Point)	2.50 (Fixed-Point) 1.25 (Floating-Point)	0.8	17.78	42710.90	9707	14.07

[†] Two 8-bit fixed-point multiplication with 32-bit fixed-point accumulation

[‡] 16-bit floating-point multiplication with 32-bit floating-point accumulation

* 1 FO4 $\approx$ 45ps, 1 NAND2 $\approx$ 4.4 μm^2 @ 90nm

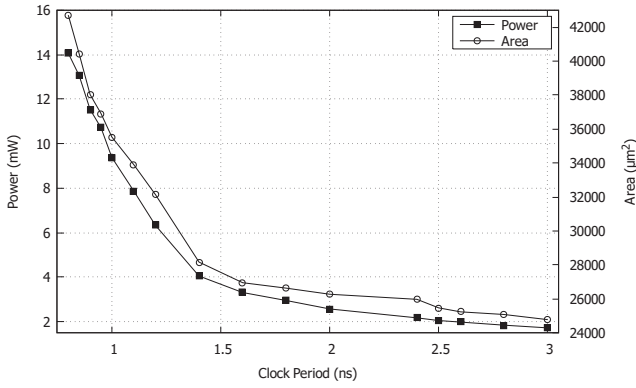


Fig. 4. Area-Delay and Power-Delay curve of the proposed design

The whole proposed architecture, including all pipeline registers, is synthesized as shown in Table II. The proposed design has a worst case delay of 0.8 ns, occupies 42710.90 μm^2 area. The power consumption of the proposed design is 14.07 mW when running with a 0.8 ns clock period. Two parallel 8-bit multiplications accumulating to 32-bit fixed-point accumulator can be accomplished in two clock cycles. One 16-bit floating-point multiplication accumulating to 32-bit floating-point accumulator can be accomplished in 3 clock cycles. The proposed architecture is fully pipelined. Each new operation can be started every clock cycle.

The area and power consumption of the proposed design under different delay requirements is shown in Fig. 4. With the delay constraint becomes larger, the area and power of the proposed design become smaller. To achieve the best performance, in this paper, the point with the smallest delay is chosen as the design point. In other cases, one can choose the appropriate design point according to the requirements.

To the best of our knowledge, there is no such fixed/floating-point merged design appeared in the literature. In order to show the advantage of our proposed design, two single-mode multiply-accumulate units are designed. One is the fixed-point multiply-accumulate unit which supports two parallel 8-bit fixed-point multiplications and accumulates the products to 32-bit fixed-point number. This has the same functionality as the proposed design in FIX mode. The other one is a floating-point multiply-accumulate unit which supports 16-

bit half-precision multiplication accumulating to 32-bit single-precision accumulator. This has the same functionality as the proposed design in FLP mode. The comparison of these two designs and our proposed design is shown in Table II. The proposed design has similar worst case delay as the floating-point multiply-accumulate. This is because the critical path is in the multiplier. The support of fixed-point multiplication does not introduce much delay overhead. In terms of area, the proposed design has 4.6% larger area. This is due to the multiplexers added to support fixed-point operations. The proposed design also consumes 3.5% more power. These overhead are negligible. However, compared to the FIX+FLP design, which is the pure combination of these two single-mode units, the proposed unit has 21% smaller area and 30.4% smaller power consumption while achieving the same functionality.

V. CONCLUSION

In this paper, a fixed/floating-point merged mixed-precision multiply-accumulate unit is designed for deep learning processors. The proposed design supports 16-bit half-precision multiplication and accumulating the product to a 32-bit floating-point accumulator. This floating-point operation can be used for deep learning training. In addition, the proposed design also supports two parallel 8-bit fixed-point multiplications and accumulating the products to a 32-bit fixed-point accumulator. This mode can be used in deep learning inference. The Karatsuba algorithm is used to divide the mantissa multiplier of half-precision multiplication. The two 8-bit multipliers generated can be used to support two parallel 8-bit fixed-point multiplications. The higher precision used in accumulation is to avoid data loss during accumulation. The proposed unit can be used in deep learning processors to enable training and high-throughput inference.

ACKNOWLEDGMENT

The authors would like to thank the Natural Sciences and Engineering Research Council of Canada (NSERC) and Department of Electrical and Computer Engineering in University of Saskatchewan for their financial support for this project.

This work was also supported by the Korean Federation of Science and Technology Societies (KOFST) grant funded by the Korean government (MSIP:Ministry of Science, ICT and Future Planning).

REFERENCES

- [1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *NATURE*, vol. 521, pp. 436–444, May 2015.
- [2] V. Sze, Y.-H. Chen, T.-J. Yang, and J. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," *arXiv preprint arXiv:1703.09039*, Mar 2017.
- [3] P. Gysel, M. Motamedi, and S. Ghiasi, "Hardware-oriented Approximation of Convolutional Neural Networks," *arXiv preprint arXiv:1604.03168*, 2016.
- [4] *Deep Learning with INT8 Optimization on Xilinx Devices*, WP486 ed., Xilinx, Apr. 2017.
- [5] P. Micikevicius, S. Narang, G. D. Jonah Alben, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed Precision Training," *arXiv preprint arXiv:1710.03740*, Oct 2017.
- [6] B. Catanzaro, "Computer Arithmetic in Deep Learning," 2016, [keynote talk], 23th IEEE Symposium on Computer Arithmetic (ARITH 23).
- [7] N. Higham, "The Rise of Multiprecision Arithmetic," 2017, [keynote talk], 24th IEEE Symposium on Computer Arithmetic (ARITH 24).
- [8] N. Brunie, F. de Dinechin, and B. de Dinechin, "A mixed-precision fused multiply and add," in *2011 Conference Record of the Forty Fifth Asilomar Conference on Signals, Systems and Computers (ASILOMAR)*, Nov 2011, pp. 165–169.
- [9] A. Karatsuba and Y. Ofman, "Multiplication of Many-Digital Numbers by Automatic Computers," in *Proceedings of the USSR Academy of Sciences*, vol. 145, 1962, pp. 293–294.
- [10] "IEEE Standard for Floating-Point Arithmetic," *IEEE Std 754-2008*, pp. 1–70, Aug 2008.
- [11] T. Lang and J. D. Bruguera, "Floating-point fused multiply-add with reduced latency," in *Proceedings. IEEE International Conference on Computer Design: VLSI in Computers and Processors*, 2002, pp. 145–150.
- [12] R. K. Montoye, E. Hokenek, and S. L. Runyon, "Design of the IBM RISC System/6000 floating-point execution unit," *IBM Journal of Research and Development*, vol. 34, no. 1, pp. 59–70, Jan 1990.
- [13] M. S. Schmookler and K. J. Nowka, "Leading zero anticipation and detection—a comparison of methods," in *Proceedings 15th IEEE Symposium on Computer Arithmetic. ARITH-15 2001*, 2001, pp. 7–12.
- [14] A. D. Booth, "A Signed Binary Multiplication Technique," *The Quarterly Journal of Mechanics and Applied Mathematics*, vol. 4, no. 2, pp. 236–240, 1951.